

1주 · 리눅스 & Docker

리눅스 커맨드 · 셸 스크립트 · Docker · Compose (5차시, 20H)

1-1 · 리눅스 커맨드 기초

텍스트 처리와 파이프라인 (1H)

1-1 · 리눅스 커맨드 기초

터미널에서 파일과 디렉터리를 다루고, 파이프를 명령을 조합해 로그에서 원하는 정보를 뽑아냅니다.

학습 목표

- 커널·셸·터미널의 역할을 구분하고 셸이 무엇인지 설명할 수 있습니다.
- pwd·ls·cd·mkdir·cp·mv·rm으로 파일과 디렉터리를 자유롭게 다룰 수 있습니다.
- chmod로 스크립트에 실행 권한을 주고 직접 실행할 수 있습니다.
- grep·sort·uniq·wc를 파이프(|)로 연결해 로그 파일을 집계할 수 있습니다.

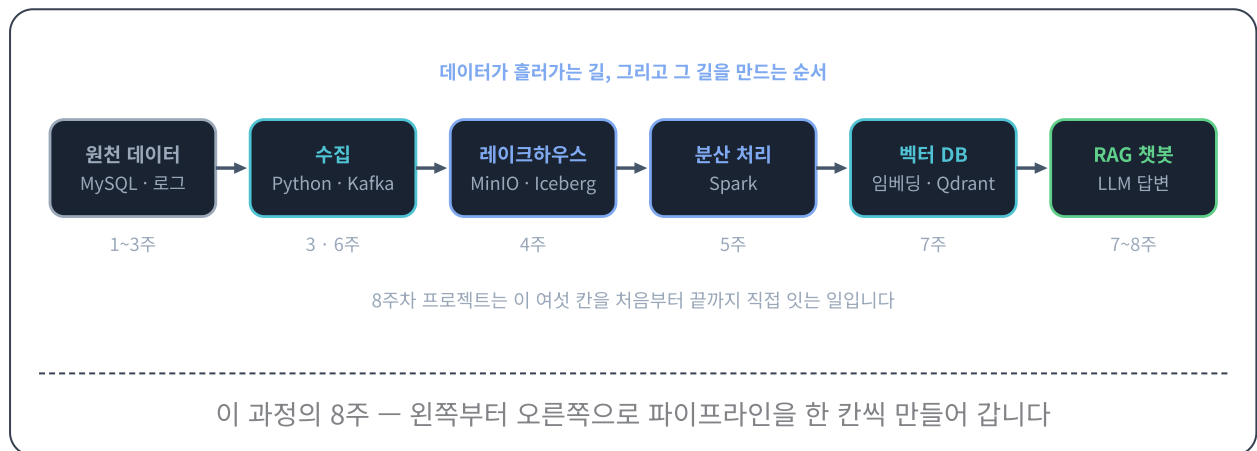
시간 배분 (4H)

구분	Chapter	시간
준비	실습 환경 준비 — Windows · macOS	0.5H
이론	리눅스와 셸의 이해	0.5H
실습	파일·디렉터리 조작 실습	1.5H
실습	텍스트 처리와 파이프라인	1H
정리·회고	커맨드 요약·체크리스트 점검·질의응답	0.5H

참고 **환경이 아직 없어도 괜찮습니다.** 오늘 첫 30분에 Windows·macOS 각각 터미널이 열리는 데까지 **함께 만들어요** (왼쪽 메뉴 「실습 환경 준비」). 오늘 필요한 것은 리눅스 명령을 칠 수 있는 터미널 하나뿐입니다. Docker 는 1-3 부터 쓰므로 그 전까지 [환경 설정] 페이지를 보고 준비하시면 됩니다.

오늘 만들 것

오늘은 이 과정 전체의 기초를 다지는 날입니다. 앞으로 8주 동안 다룰 도구 — Docker · Spark · Kafka · MinIO — 가 모두 리눅스 위에서 돌아갑니다. 그리고 그 위에서 하는 일도 오늘 배우는 것과 크게 다르지 않습니다. 작은 명령을 이어 붙여 데이터를 한 방향으로 흘러보내는 일입니다.



왼쪽 메뉴에서 챕터를 하나씩 따라가시면 됩니다. 첫 30분에 실습 환경을 만들고, 이론 30분으로 "명령을 치면 안에서 무슨 일이 일어나는지"를 잡고, 실습 2.5시간으로 손에 익히고, 마지막 30분에 정리합니다. **개강일이라 이론은 핵심만 봤습니다** — 못 다룬 부분은 학습 노트에 그대로 남아 있으니 나중에 읽으셔도 됩니다.

실습 환경 준비 — Windows · macOS (0.5H)

오늘 실습에 필요한 것은 리눅스 명령을 칠 수 있는 터미널 하나입니다. Windows 와 macOS 를 나란히 놓았으니, 쓰시는 쪽 안내를 따라 실습해 주시면 됩니다.

오늘 배울 것은 터미널입니다

오늘은 리눅스 명령을 손에 익히는 날입니다. 그러려면 **리눅스 명령이 통하는 터미널**이 하나 있어야 합니다. 그것만 되면 오늘 실습은 끝까지 갑니다.

macOS 는 이미 유닉스라 **터미널을 칠만 해도 됩니다.** Windows 는 리눅스가 아니므로 **WSL2도** **유분부를 찾아** 리눅스 셸을 만듭니다. 갈리는 것은 여기까지이고, 그 뒤 명령은 양쪽이 같습니다.

참고 Docker 는 오늘 쓰지 않습니다. 1-3(수요일)부터 쓰므로 그 전까지 [환경 설정] 페이지를 보고 준비하시면 됩니다. 오늘 Docker 설치가 안 끝나도 실습에 지장이 없습니다.

Windows 노트북

WSL2 는 Windows 안에서 진짜 리눅스 커널을 돌리는 기능입니다. 가상머신처럼 무겁지 않고, Windows 와 파일을 주고받을 수 있습니다.

1단계. 시작 메뉴에서 **PowerShell** 을 마우스 오른쪽 버튼으로 눌러 **관리자 권한으로 실행** 합니다. 일반 실행으로는 설치가 되지 않습니다.

powershell

```
1 wsl --install -d Ubuntu-22.04
```

확인 포인트: 설치가 끝나면 **재부팅하라는 안내**가 나옵니다. 재부팅해야 다음 단계로 갑니다.

2단계. 재부팅하면 우분투 창이 저절로 뜹니다. **사용자 이름과 비밀번호**를 만듭니다.

주의 비밀번호를 칠 때 화면에 아무것도 나타나지 않습니다. 고장이 아니라 리눅스가 원래 그렇게 동작합니다 — 그냥 치고 Enter 를 누르세요. 이 비밀번호는 나중에 sudo 를 쓸 때마다 필요하니 기억해 두십시오.

3단계. 제대로 올라갔는지 확인합니다. PowerShell 에서 칩니다.

powershell

```
1 wsl -l -v
```

확인 포인트: **NAME** 이 **Ubuntu-22.04** , **VERSION** 이 **2** 로 나오면 됩니다. VERSION 이 1 이면 WSL1 이라 뒤에서 Docker 가 안 붙습니다 — **wsl --set-version Ubuntu-22.04 2** 로 올리세요.

4단계. 앞으로 실습에 쓸 터미널을 엽니다. 시작 메뉴에서 **Ubuntu** 를 찾아 실행하거나, **Windows 터미널** 을 열고 탭에서 Ubuntu 를 고릅니다.

macOS · 맥북

macOS 는 유닉스 계열이라 따로 설치할 것이 없습니다. **터미널을 열면 그대로 실습할 수 있습니다.**

1단계: `Command + Space` 를 눌러 스포트라이트를 열고 **터미널**을 칩니다. (응용 프로그램 → 유틸리티 → 터미널 에도 있습니다)

2단계: 지금 쓰는 셸이 무엇인지 봅니다.

```
bash
```

```
1 echo $SHELL
```

참고 포인트: 최근 macOS 는 `/bin/zsh` 가 나옵니다. 오래된 기기는 `/bin/bash` 일 수 있는데, **오늘 쓰는 명령은 양쪽에서 똑같이 동작하니 그대로 두셔도 됩니다.**

Windows · macOS 공통

여기서부터는 Windows(우분투 터미널)와 macOS(터미널)가 **같습니다**. 아래 세 줄을 차례로 쳐 보세요.

bash

```
1 # 1) 내가 누구인가
2 whoami
3
4 # 2) 지금 어디에 있는가
5 pwd
6
7 # 3) 이 시스템이 무엇인가
8 uname -s
```

확인 포인트: `whoami` 는 내가 만든 사용자 이름, `pwd` 는 `/home/사용자명` (Windows·WSL) 또는 `/Users/사용자명` (macOS), `uname -s` 는 `Linux` (WSL) 또는 `Darwin` (macOS) 이 나오면 준비가 끝난 것입니다.

마지막으로 **오늘 실습에 쓸 폴더**를 만들어 둡니다. 8주 내내 이 아래에서 작업합니다.

bash

```
1 mkdir -p ~/data-eng/day01
2 cd ~/data-eng/day01
3 pwd
```

확인 포인트: `pwd` 가 `/home/사용자명/data-eng/day01` (또는 `/Users/...`)로 나오면 됩니다. 여기까지 되면 다음 챕터로 넘어가셔도 좋습니다.

주의 Windows 사용자는 실습 파일을 반드시 리눅스 홈(~/) 안에 두세요. `/mnt/c` 아래(윈도우 디스크)에서 작업하면 파일 접근이 몇 배 느리고, 권한과 줄바꿈 문제로 뒤에서 스크립트가 죽습니다.

막혔을 때 — 첫날 가장 자주 나오는 것

증상	원인	해결
<code>wsl --install</code> 이 「가상화가 사용 안 함」이라고 함	BIOS 에서 CPU 가상화가 꺼져 있음	재부팅 → BIOS 진입(보통 F2·DEL) → intel 은 VT-x, AMD 는 SVM 을 켜다
설치는 됐는데 우분투 창이 안 뜸	재부팅을 아직 안 함	재부팅 후 시작 메뉴에서 Ubuntu 를 실행
<code>wsl -l -v</code> 에서 VERSION 이 1	WSL1 로 설치됨	<code>wsl --set-version Ubuntu-22.04 2</code> (몇 분 걸립니다)
비밀번호를 쳐도 화면에 안 보임	리눅스의 정상 동작	그대로 치고 Enter — 고장이 아닙니다
macOS 에서 <code>command not found</code>	오타이거나 아직 설치 안 된 명령	오늘 쓰는 명령은 전부 기본 포함입니다. 설치와 띄어쓰기 를 먼저 보세요
설치가 회사 노트북 정책에 막힘	관리자 권한·보안 정책	수업 중에 바로 알려 주세요. 할리우드 데제 해 오늘 실습은 따라올 수 있게 해 드립니다

TIP 설치가 오래 걸리더라도 조급해하지 마세요. 첫날은 원래 환경을 만드는 데 시간이 걸립니다. 막히면 바로 손을 들어 주시고, 해결되면 트러블슈팅 위키(패들릿)에 증상·원인·해결 세 줄로 남겨 주시면 다음 사람이 훨씬 빨리 지나갑니다.

리눅스와 셸의 이해 (0.5H)

명령을 한 줄 입력했을 때 컴퓨터 안에서 무슨 일이 벌어지는지부터 봅니다. 여기를 잡아 두면 뒤에 나오는 오류 메시지가 대부분 읽힙니다.

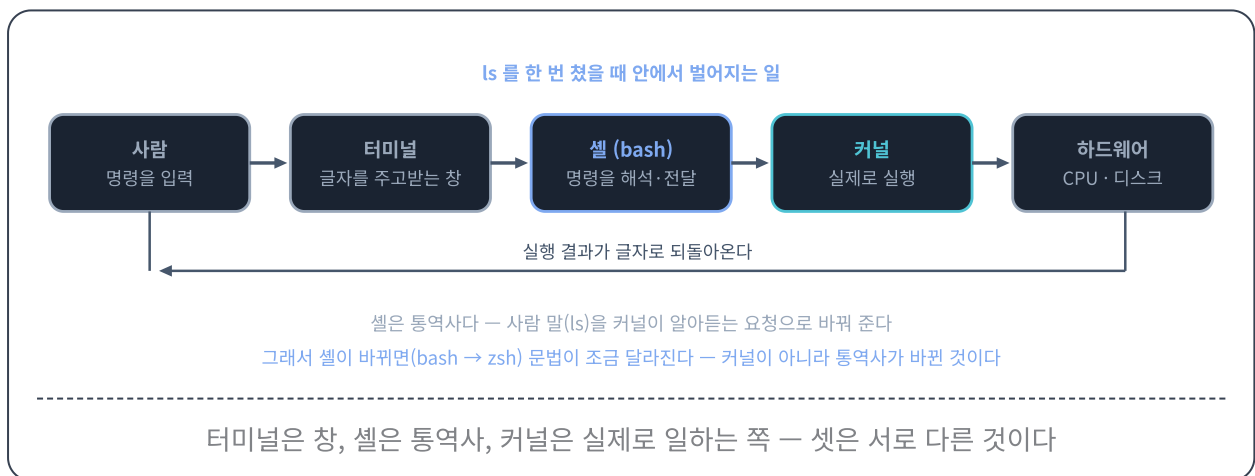
왜 리눅스부터인가

데이터 엔지니어가 다루는 것은 대부분 남의 컴퓨터입니다. 클라우드 서버도, 컨테이너 안도, Spark 클러스터의 워커도 전부 리눅스입니다. 그리고 그 안에는 마우스로 누를 화면이 없습니다. 터미널이 유일한 출입구라서, 터미널을 다루는 능력이 곧 일할 수 있는 범위가 됩니다.

참고 맥을 쓰신다면 이미 절반은 하신 겁니다. macOS는 Unix 계열이라 오늘 배우는 명령의 대부분이 그대로 통합니다. Windows는 WSL2(Ubuntu) 터미널을 여시면 진짜 리눅스와 같은 환경이 됩니다.

커널·셸·터미널 — 셋은 서로 다른 것입니다

많은 분이 이 셋을 뭉뚱그려 "터미널"이라고 부르는데, 나눠서 알아 두면 나중에 "zsh에서는 되는데 bash에서는 안 된다" 같은 상황을 바로 이해하게 됩니다.



- 커널(Kernel): 운영체제의 심장입니다. CPU를 누구에게 얼마나 줄지, 디스크의 어느 자리에 쓸지를 실제로 결정합니다. 우리는 커널에게 직접 말을 걸지 않습니다.
- 셸(Shell): 사람의 명령을 커널이 알아듣는 요청으로 바꿔 주는 통역사입니다. bash · zsh · sh 가 전부 셸의 종류입니다. 통역사가 바뀌면 말투(문법)가 조금 달라집니다.
- 터미널(Terminal): 글자를 보여 주고 입력받는 창일 뿐입니다. iTerm · Windows Terminal · VS Code 통합 터미널이 전부 여기 해당합니다. 창을 바꿔도 안에서 도는 셸은 그대로입니다.
- 한 줄로: 터미널이라는 창 안에서 셸이 돌고 있고, 셸이 커널에게 일을 시킵니다.

지금 내 셸이 무엇인지 확인해 봅시다

bash

```
1 # 지금 쓰고 있는 셸
2 echo $SHELL
3
4 # 이 컴퓨터에 설치된 셸 목록
5 cat /etc/shells
6
7 # bash 버전 (WSL2 우분투는 보통 5.x)
8 bash --version | head -n 1
```

확인 포인트: macOS는 보통 /bin/zsh, WSL2 우분투는 /bin/bash 가 나옵니다. 둘 중 무엇이 나와도 정상이며, 이번 과정의 실습은 양쪽 모두에서 동작합니다.

TIP 앞으로 인터넷에서 스크립트를 복사해 올 때 첫 줄의 #!/bin/bash 를 자주 보게 됩니다. 이건 "이 파일은 bash 통역사로 읽어 달라"는 표시입니다. 셸의 종류를 알아 뒤야 이 줄이 눈에 들어옵니다.

파일시스템은 하나의 큰 서랍장입니다

Windows는 C 드라이브 · D 드라이브처럼 저장장치마다 최상위가 따로 있지만, 리눅스는 최상위가 / 하나뿐입니다. 새 디스크를 꽂아도 그 서랍장 어딘가의 칸으로 들어옵니다. 처음에는 낯설지만 익숙해지면 훨씬 단순합니다.

위치	무엇이 들어 있나	데이터 엔지니어가 여기를 볼 때
/	서랍장 전체의 맨 위(루트)	경로가 / 로 시작하면 여기서부터 읽습니다
/home/ 사용자명	내 방. 줄여서 ~ 로 씁니다	실습 파일은 전부 여기에 만듭니다
/etc	설정 파일 보관함	서비스 설정을 고칠 때
/var/log	계속 쌓이는 것들 — 특히 로그	장애가 나면 가장 먼저 열어 보는 곳
/tmp	임시 파일. 재부팅하면 사라집니다	중간 산출물을 잠깐 둘 때
/usr/bin	설치된 명령들의 실체	which 로 찾으려면 대개 여기를 가리킵니다

주의 WSL2를 쓰시는 분은 /mnt/c 로 Windows 디스크에 접근할 수 있습니다. 다만 그 경로에서 작업하면 파일 입출력이 매우 느립니다. 실습은 반드시 리눅스 홈(~/) 안에서 하세요. 3~4주차에 큰 파일을 다룰 때 속도 차이가 크게 납니다.

절대 경로와 상대 경로

- 절대 경로: / 로 시작하는, 처음부터 끝까지 쓴 주소입니다. "서울시 ○○구 ○○로 1"에 해당합니다.
예) /home/user/data-eng/week1
- 상대 경로: 지금 서 있는 자리를 기준으로 한 주소입니다. "여기서 한 블록 앞"에 해당합니다. 예)
./logs · ../week2
- . 은 현재 디렉터리, .. 은 한 단계 위, ~ 는 내 홈 디렉터리를 뜻합니다.
- 스크립트 안에서는 절대 경로를 쓰는 편이 안전합니다. 어디서 실행하든 같은 곳을 가리키기 때문입니다.

bash

```

1 # 지금 어디에 서 있는지
2 pwd
3
4 # 절대 경로로 이동
5 cd /var/log
6 pwd
7
8 # 상대 경로로 한 단계 위
9 cd ..
10 pwd
11
12 # 언제든지 내 홈으로 (인자 없는 cd 도 같은 뜻)
13 cd ~
14 pwd

```

확인 포인트: cd /var/log 후 pwd 가 /var/log, cd .. 후 /var, cd ~ 후 /home/사용자명(macOS는 /Users/사용자명)을 출력하면 제대로 따라오신 겁니다.

권한(rwx) 읽는 법

ls -l 을 치면 맨 왼쪽에 -rwxr-xr-x 같은 열 글자가 나옵니다. 처음 보면 암호 같지만 규칙은 아주 단순합니다. 앞의 한 글자는 종류이고, 나머지 아홉 글자를 세 개씩 끊어 읽습니다.

자리	표기 예	뜻
1번째	- 또는 d	- 는 일반 파일, d 는 디렉터리
2~4번째	rwx	소유자(나)가 읽기·쓰기·실행 가능
5~7번째	r-x	같은 그룹은 읽기·실행만 가능(쓰기 불가)
8~10번째	r-x	그 외 모든 사용자도 읽기·실행만 가능

TIP 스크립트가 Permission denied 로 안 돌아가면 열에 아홉은 x(실행) 권한이 없어서입니다. chmod +x 파일명 한 줄이면 해결됩니다. 이건 오늘 실습에서 직접 겪어 보게 됩니다.

이 1시간에서 꼭 가져갈 것

- 터미널·셸·커널은 다른 것이고, 내가 명령을 주는 상대는 셸이다.

- 리눅스의 최상위는 / 하나뿐이고, 내 작업은 ~ 안에서 한다.
- 경로는 절대(/로 시작)와 상대(./ ../)가 있고, 헛갈리면 pwd 로 현재 위치를 먼저 확인한다.
- 권한 열 글자는 [종류][소유자][그룹][기타] 로 끊어 읽는다.

파일·디렉터리 조작 실습 (1.5H)

앞으로 8주 내내 쓸 실습 폴더를 직접 만들고, 파일을 만들고 옮기고 지우고 권한을 바꿔 봅니다. 여섯 단계 모두 직접 손으로 쳐 보시면 좋습니다.

참고 이 챕터에서 만드는 ~/data-eng 폴더는 8주 내내 씁니다. 지우지 마세요. 각 주차 폴더 (week1 ~ week8)를 그 안에 만들어 갑니다.

1단계 — 현재 위치를 확인하고 이동해 봅니다

bash

```
1 # 현재 위치(절대 경로) 확인
2 pwd
3
4 # 숨김 파일 포함 상세 목록
5 ls -al
6
7 # 루트로 갔다가 홈으로 돌아오기
8 cd /
9 pwd
10 cd ~
11 pwd
```

확인 포인트: 첫 pwd는 현재 위치의 절대 경로를, cd / 후에는 /, cd ~ 후에는 /home/사용자명 (macOS는 /Users/사용자명)을 출력하면 성공입니다.

TIP ls -al 에서 . 으로 시작하는 파일이 숨김 파일입니다. .bashrc · .gitconfig 처럼 설정 파일이 대부분 여기 해당합니다. -a 를 빼면 안 보입니다.

2단계 — 8주 동안 쓸 실습 폴더를 만듭니다

bash

```
1 # 중간 디렉터리까지 한 번에 생성(-p)
2 mkdir -p ~/data-eng/week1/logs
3 cd ~/data-eng/week1
4 pwd
5
6 # 구조를 눈으로 확인
7 ls -R ~/data-eng
```

확인 포인트: pwd 가 ~/data-eng/week1 을 가리키고, ls -R 에 logs 디렉터리가 보이면 성공입니다.

-p 옵션이 하는 일: 중간 단계 폴더가 없어도 알아서 함께 만들어 줍니다. -p 없이 mkdir ~/data-eng/week1/logs 를 치면 data-eng 가 없다면 실패합니다.

3단계 — 파일을 만들고 복사·이동·이름 변경합니다

bash

```
1 cd ~/data-eng/week1
2
3 # 빈 파일 생성
4 touch notes.txt
5
6 # 복사 (원본은 그대로 남습니다)
7 cp notes.txt notes_backup.txt
8 ls
9
10 # logs/ 로 이동 (원본이 사라집니다)
11 mv notes_backup.txt logs/
12 ls
13 ls logs
14
15 # 이름 변경도 mv 로 합니다 (같은 자리에서 옮기는 셈)
16 mv notes.txt memo.txt
17 ls
```

확인 포인트: 마지막 ls 에 memo.txt 가 보이고 notes.txt 는 없어야 합니다. logs 안에는 notes_backup.txt 가 있어야 합니다.

TIP cp 와 mv 의 차이는 원본을 남기느냐입니다. 그리고 리눅스에는 "이름 바꾸기" 명령이 따로 없습니다 — 같은 폴더 안에서 mv 하는 것이 곧 이름 변경입니다.

4단계 — 삭제, 그리고 안전장치

리눅스의 rm 에는 휴지통이 없습니다. 지우면 그 자리에서 사라집니다. 그래서 이 단계는 특히 천천히 따라와 주세요.

bash

```

1 cd ~/data-eng/week1
2
3 # 연습용 파일·폴더 만들기
4 touch trash1.txt trash2.txt
5 mkdir -p trashdir
6 ls
7
8 # 하나씩 확인하며 삭제 (-i: 물어봅니다. y 를 입력)
9 rm -i trash1.txt
10
11 # 그냥 삭제
12 rm trash2.txt
13
14 # 폴더는 -r(재귀) 이 필요합니다
15 rm -r trashdir
16 ls

```

확인 포인트: 마지막 ls 에 trash 로 시작하는 것이 하나도 남아 있지 않고, memo.txt 와 logs 만 보이면 성공입니다.

주의 rm -rf / 는 절대 치지 마세요. 시스템 전체를 지우려 드는 명령입니다. 그리고 rm -rf 를 쓸 때는 실행 전에 경로를 소리 내어 읽어 보는 습관을 들이시길 권합니다. 특히 변수를 쓴 rm -rf "\$DIR"/ 형태에서 변수가 비면 rm -rf / 가 됩니다.

5단계 — 스크립트를 만들고 실행 권한을 부여합니다

bash

```

1 echo '#!/bin/bash' > hello.sh
2 echo 'echo Hello, Data Engineering!' >> hello.sh
3
4 # 내용 확인
5 cat hello.sh
6
7 # 실행 권한 부여 전 - 여기서 한 번 실패해 보는 것이 목적입니다
8 ls -l hello.sh
9 ./hello.sh
10
11 # 권한을 주고 다시 실행
12 chmod +x hello.sh
13 ls -l hello.sh
14 ./hello.sh

```

확인 포인트: chmod 전에는 Permission denied 가 나와 정상입니다. chmod 후 권한이 -rwxr-xr-x 로 바뀌고 Hello, Data Engineering! 이 출력되면 성공입니다.

> 와 >> 의 차이: > 는 파일을 덮어쓰고, >> 는 뒤에 이어 붙입니다. 위에서 첫 줄에 > 를 쓰고 둘째 줄에 >> 를 쓴 이유가 이것입니다. 둘 다 > 로 썼다면 두 번째 줄만 남았을 겁니다.

6단계 — 찾고 세어 봅니다

bash

```
1 cd ~/data-eng
2
3 # 이름으로 찾기
4 find . -name "*.sh"
5
6 # 디렉터리만 / 파일만
7 find . -type d
8 find . -type f
9
10 # 파일 개수 세기 (find 결과를 wc 로 넘깁니다 - 다음 챕터의 예고편입니다)
11 find . -type f | wc -l
12
13 # 용량 확인
14 du -sh ~/data-eng
```

확인 포인트: find . -name "*.sh" 에 ./week1/hello.sh 가 나오고, find . -type f | wc -l 이 1 이상의 숫자를 출력하면 성공입니다.

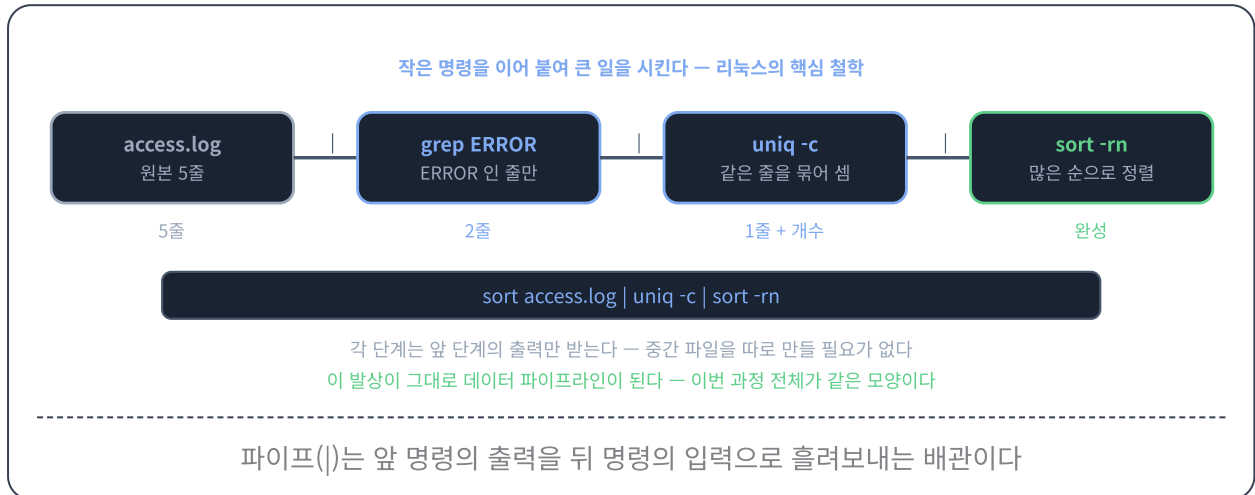
여기까지 되면 다음으로

- ~/data-eng/week1 과 그 안의 logs 폴더가 만들어져 있다.
- memo.txt 와 hello.sh 가 week1 안에 있다.
- ./hello.sh 를 실행하면 인사말이 출력된다.
- find 로 파일을 찾고 wc -l 로 개수를 셀 수 있다.

텍스트 처리와 파이프라인 (1H)

작은 명령을 파이프(|)로 이어 붙여 로그에서 원하는 숫자를 뽑아냅니다. 이번 과정 전체가 이 발상의 확장판입니다.

데이터 엔지니어가 실무에서 가장 많이 하는 일 중 하나가 "로그를 열어 원인을 찾는 것"입니다. 로그는 보통 수십만 줄이라 눈으로 볼 수 없고, 필요한 것만 걸러 세어 봐야 합니다. 그 도구가 오늘 배우는 `grep` · `sort` · `uniq` · `wc` 이고, 이들을 잇는 것이 파이프입니다.



1단계 — 실습용 로그 파일을 만듭니다

bash

```

1 cd ~/data-eng/week1
2
3 # EOF 까지 통째로 복사해 붙여넣으세요
4 cat > access.log << 'EOF'
5 INFO login user=kim
6 ERROR db timeout
7 INFO login user=lee
8 ERROR db timeout
9 INFO logout user=kim
10 WARN slow query 1200ms
11 ERROR db timeout
12 INFO login user=park
13 EOF
14
15 # 잘 만들어졌는지 확인
16 cat access.log
17 wc -l access.log
  
```

확인 포인트: `wc -l` 결과가 8 이면 성공입니다. 숫자가 다르면 붙여넣기가 중간에 잘린 것이니 파일을 지우고(`rm access.log`) 다시 만드세요.

TIP cat > 파일명 << 'EOF' 형태를 히어독(here document)이라고 합니다. EOF 라고 친 줄이 나올 때까지의 내용을 그대로 파일에 씁니다. 여러 줄을 한 번에 만들 때 아주 편해서 앞으로 자주 쓰게 됩니다.

2단계 — grep 으로 원하는 줄만 골라냅니다

bash

```
1 # ERROR 가 들어간 줄만
2 grep ERROR access.log
3
4 # 개수만 (-c)
5 grep -c ERROR access.log
6
7 # 대소문자 무시 (-i)
8 grep -i error access.log
9
10 # ERROR 가 아닌 줄만 (-v: 반대로)
11 grep -v ERROR access.log
12
13 # 줄 번호까지 (-n)
14 grep -n ERROR access.log
```

확인 포인트: grep -c ERROR 가 3, grep -v ERROR 는 5줄이 나오면 성공입니다. $3 + 5 = 8$ 로 전체 줄 수와 맞습니다.

3단계 — 파이프로 이어 붙입니다

여기가 오늘의 핵심입니다. 앞 명령의 출력이 파일에 저장되지 않고 곧바로 뒤 명령의 입력으로 흘러갑니다.

bash

```
1 # 같은 줄끼리 묶어 개수를 세고, 많은 순으로 정렬
2 # sort : 같은 줄을 이웃하게 모읍니다 (uniq 는 이웃한 것만 묶습니다)
3 # uniq -c : 묶으면서 개수를 앞에 붙입니다
4 # sort -rn : 숫자(n) 기준 내림차순(r)
5 sort access.log | uniq -c | sort -rn
6
7 # 한 단계씩 끊어 보면 무슨 일이 일어나는지 보입니다
8 sort access.log
9 sort access.log | uniq -c
```

확인 포인트: 마지막 명령의 첫 줄이 "3 ERROR db timeout" 이면 성공입니다. 중간 단계를 하나씩 실행해 보시면 sort 가 왜 먼저 필요한지 눈으로 확인됩니다.

주의 uniq 는 이웃한 중복만 묶습니다. sort 없이 uniq -c 만 쓰면 떨어져 있는 같은 줄이 따로따로 세어집니다. 실무에서 집계가 이상하게 나오는 흔한 원인이니 sort | uniq -c 를 한 덩어리로 외워 두세요.

4단계 — 잘라내기와 앞뒤만 보기

bash

```
1 # 앞 3줄 / 뒤 2줄
2 head -n 3 access.log
3 tail -n 2 access.log
4
5 # 공백으로 잘라 두 번째 칸만 (-d 구분자, -f 몇 번째)
6 cut -d ' ' -f 2 access.log
7
8 # 로그 레벨(첫 칸)만 뽑아 집계
9 cut -d ' ' -f 1 access.log | sort | uniq -c | sort -rn
```

확인 포인트: 마지막 명령이 INFO 4 / ERROR 3 / WARN 1 세 줄을 개수와 함께 출력하면 성공입니다.

5단계 — 미니 과제: 사용자별 접속 횟수

access.log 에서 user=이름 부분만 뽑아 사용자별로 몇 번 나왔는지 세어 보세요. 아래 힌트를 참고하되, 먼저 스스로 조합해 보시면 좋습니다.

- 힌트 1: user= 가 들어간 줄만 남기려면 grep 을 씁니다.
- 힌트 2: 공백으로 잘라 마지막 칸만 뽑으려면 cut -d ' ' -f 3 을 씁니다.
- 힌트 3: 세는 것은 3단계에서 쓴 sort | uniq -c | sort -rn 그대로입니다.

bash

```
1 # 정답 예시 - 먼저 직접 해 보신 다음에 펼쳐 보세요
2 grep 'user=' access.log | cut -d ' ' -f 3 | sort | uniq -c | sort -rn
3
4 # 결과
5 #   2 user=kim
6 #   1 user=lee
7 #   1 user=park
```

확인 포인트: kim 이 2회로 가장 위에 오면 성공입니다. 결과를 파일로 남기려면 뒤에 > user_count.txt 를 붙이면 됩니다.

TIP 방금 만든 것이 아주 작은 데이터 파이프라인입니다. 원본을 읽어(cat) 걸러내고(grep) 모양을 바꾸고(cut) 집계해(uniq -c) 정렬한(sort) 다음 저장(>)했습니다. 5주차 Spark에서 하는 일도 규모만 다를 뿐 순서는 똑같습니다.

정리·회고 (0.5H)

오늘 쓴 명령을 한 장으로 모으고, 자주 겪는 오류 세 가지를 미리 짚습니다.

오늘의 커맨드 요약

하려는 일	명령	자주 쓰는 옵션
지금 어디인지	pwd	—
목록 보기	ls	-a 숨김 포함 · -l 상세 · -R 하위까지
이동	cd	cd ~ 홈 · cd .. 한 단계 위 · cd - 직전 위치
폴더 만들기	mkdir	-p 중간 폴더까지 함께
복사 / 이동·이름변경	cp / mv	-r 폴더째
삭제	rm	-i 물어보기 · -r 폴더째 (휴지통 없음)
권한 주기	chmod	+x 실행 권한
내용 보기	cat / head / tail	-n 줄 수 지정
찾기	grep / find	grep -i -v -c -n · find -name -type
정렬·집계	sort / uniq / wc	sort -rn · uniq -c · wc -l

오늘의 체크리스트

- 터미널·셸·커널의 차이를 한 문장으로 말할 수 있다.
- pwd·cd·ls로 원하는 디렉터리에 정확히 찾아갈 수 있다.
- mkdir -p로 실습 디렉터리 구조(~/data-eng/week1/logs)를 만들었다.
- chmod +x로 권한을 바꿔 hello.sh를 직접 실행했다.
- 파이프를 써서 access.log의 ERROR 라인 수를 집계했다.
- 사용자별 접속 횟수 미니 과제를 스스로 조합해 풀었다.

자주 겪는 오류

주의 증상: command not found → **원인:** 명령 철자가 틀렸거나 해당 명령이 설치되지 않았습니다. → **해결:** 철자를 다시 확인하고, which 명령어이름 으로 설치 여부를 확인합니다. WSL2에서는 sudo apt update 후 sudo apt install 패키지명 으로 설치합니다.

주의 증상: ./hello.sh 실행 시 Permission denied → **원인:** 파일에 실행(x) 권한이 없습니다. → **해결:** chmod +x hello.sh 로 실행 권한을 부여한 뒤 다시 실행합니다. ls -l 로 권한에 x가 생겼는지 확인합니다.

주의 증상: No such file or directory → **원인:** 현재 위치가 파일이 있는 디렉터리가 아니거나 상대 경로가 틀렸습니다. → **해결:** pwd로 현재 위치를 확인하고 ls로 파일이 실제로 있는지 확인한 뒤, cd로 이동하거나 절대 경로로 다시 시도합니다.

예상 질문

- Q. 맥이랑 우분투랑 명령이 다른가요? — 90%는 같습니다. 다만 sed 나 date 처럼 옵션이 다른 것이 몇 개 있습니다. 실습에서 만나면 그때 짚어 드립니다.
- Q. 명령을 다 외워야 하나요? — 아닙니다. 오늘 요약표의 10개 정도만 손에 익히면 나머지는 필요할 때 찾아 쓰면 됩니다. man 명령어 로 설명을 볼 수 있습니다.
- Q. 실수로 파일을 지웠는데 복구할 수 있나요? — 리눅스 rm 은 복구가 안 됩니다. 그래서 중요한 작업 전에는 cp 로 백업을 먼저 뜨는 습관이 필요합니다.
- Q. 화살표 위 키를 누르면 이전 명령이 나오는데 더 편한 방법이 있나요? — history 로 목록을 보고, Ctrl+R 로 이전 명령을 검색할 수 있습니다. 실무에서 아주 자주 씁니다.

심화 과제 (선택)

- find ~/data-eng -name "*.log" 로 로그 파일을 찾고, 찾은 파일의 라인 수를 wc -l로 세어 보세요.
- ls -al 출력의 각 열(권한·링크 수·소유자·그룹·크기·수정시각·이름)이 무엇을 뜻하는지 한 줄씩 정리해 보세요.
- access.log 에서 WARN 과 ERROR 를 함께 뽑아 보세요. (힌트: grep -E "WARN|ERROR")
- 오늘 만든 집계 명령을 hello.sh 처럼 스크립트 파일로 저장해 두세요. 내일(1-2) 그 스크립트를 발전시킵니다.

참고 링크

- GNU Coreutils 매뉴얼: <https://www.gnu.org/software/coreutils/manual/coreutils.html>
- GNU Bash 매뉴얼: <https://www.gnu.org/software/bash/manual/>

이번 주 산출물

제출은 [산출물 제출](#) 페이지에서 하고, 제출 후 [주차별 산출물 갤러리](#)의 해당 주 섹션에 [1주] 이름 - 산출물명 형식으로 올려 동료와 공유해 주세요.

Built with the LIVE method by 이애본 (Aebon Lee) · DreamIT Biz